

Developer Guideline

Deployment Cycle Guide & Checklist

*End-to-end guide for developers across the deployment cycle:
repository setup, CI pipeline, containerization, infrastructure, and production
readiness.*

Companion to the DevOps Engagement & Governance Policy.

Date: May 25, 2026

Version: 1.5

Table of Contents

Note: if the table below is empty, right-click on it and choose **Update Field** (or select the document with Ctrl+A and press F9). Word does not auto-populate this field on first open.

Table of Contents	2
How to Use This Guide	3
Branching model used in this document	3
Layout	3
Stage tags in descriptions	3
Terminology	4
Exceptions	4
Section 1 — Universal Deployment Cycle	5
1.1 Repository & Branching	5
1.2 Documentation	5
1.3 Secrets & Configuration	6
1.4 Containerization (Mandatory)	6
1.5 Observability	7
1.6 Security Baseline	7
1.7 Testing & Coverage	8
1.8 CI/CD Pipelines — Triggers & Stages	8
Application repository pipeline	8
Application repo CI requirements	9
1.9 Pre-commit Hooks	10
1.10 Deployment Readiness	10
1.11 Infrastructure — Terraform Repository	11
Repository layout	11
Terraform repo CI/CD	11
Terraform repo requirements	12
Section 2 — Python / Django Addendum	14
Section 3 — Node.js / React Addendum	15
Section 4 — Java / Spring Addendum	16
Section 5 — Other Stacks (Generic Addendum)	17
Section 6 — Production Hardening	18
Appendix A — Code & Config Reference	19
A.1 Environment Variable Validation (Python)	19
A.2 Multi-stage Dockerfile (Python)	19
A.3 Gunicorn Production Config	20
A.4 Multi-stage Dockerfile (Node.js)	20
A.5 .dockerignore	20
A.6 .pre-commit-config.yaml	21
A.7 Trivy — Dependency & Image Scanning	22

A.8 Health vs Readiness Endpoints	22
A.9 Graceful Shutdown (Node.js)	23
A.10 Structured Logging to STDOUT (Django)	23
A.11 Terraform environment/main.tf (example)	23
A.12 Terraform backend.tf (example)	24
Appendix B — Per-Gate Quick Map	25
Gate 4 (Build & Provision) — verify on first pipeline run	25
Gate 5 (Pre-Production Readiness) — verify before prod cutover	25
Gate 6 (Go-Live) — verify on go-live call	25
Document Control	27
Version History	27
Review Cadence	28

How to Use This Guide

This is the developer-facing companion to the *DevOps Engagement & Governance Policy*. It is a working guideline for the **entire deployment cycle** — not only the moment before promotion. Open it when starting a project (repository setup, branching, CI), refer back to it through development (containerization, observability, security), and re-read it before every promotion (develop → UAT → main).

The policy defines *what* is required and *who* enforces it across the SDLC gates; this guide defines *how* developers satisfy it day-to-day.

Branching model used in this document

Branch	Environment	Notes
develop	Dev	Integration branch. Feature branches PR into develop. Merge auto-deploys to dev.
uat	Staging / UAT	Promoted from develop via PR. Merge auto-deploys to UAT.
main	Production	Promoted from uat via PR. Merge does not auto-deploy — deploy is a manual approval step in the pipeline.

Promotion flow: **feature/*** → **develop** → **uat** → **main**. Hotfixes branch from main, merge to main, and are back-merged to uat and develop.

Layout

- **Sections 1.1–1.11** are the universal cycle — every project follows them regardless of stack.
- **Sections 2–5** are stack-specific addendums (Python / Django, Node.js / React, Java / Spring, other). They *add to* Section 1, not replace it.
- **Section 6** is production hardening — required for medium / large projects and any project handling sensitive data.
- **Appendix A** contains code and config snippets referenced from items.
- **Appendix B** maps items to the policy gates (G4, G5, G6) for DevOps reviewers.

Stage tags in descriptions

By default, every checklist item applies in every environment (dev, UAT, and prod). A small number of items only become relevant at production — those carry an inline colored tag at the end of their description:

- **[Prod only]** — required only before promoting to production. Typically things that need real production data or recent verification (e.g., restore tested in the last 30 days).
- **[Per project plan]** — stage and depth defined in the project's own plan; the document sets a minimum floor.

Items without any tag apply universally. A tag never means an item *stops* being required after the named stage — it means that's the *first* point at which it becomes a hard gate. The

previously-used *[UAT onwards]* tag is intentionally not used: in practice, almost everything you would want at UAT is also worth having from dev day one, so universal coverage is the default.

Terminology

A few terms used throughout this document, in plain language:

- **Multi-environment config** — the same code image runs in dev, UAT, and prod by reading environment-specific values at startup from env vars or a secret manager. No environment values are hardcoded into the image.
- Example: `DJANGO_SETTINGS_MODULE=settings.prod` plus `DATABASE_URL` supplied by the prod secret manager.
- **Env validation at startup** — a small script (e.g., `validate_env.py`) that runs when the container starts and fails fast with a clear error if any required env var is missing or malformed. Stops the container starting in a half-broken state.
- **AuthN / AuthZ** — Authentication (proving who you are: login, tokens, certs) AND Authorization (proving you are allowed to do this specific thing: role / permission checks). Both must be enforced on every protected endpoint, not just at login.
- **Trunk-based with short-lived branches** — we do not use this model. We use a `develop / uat / main` long-lived branch model (described above). Mentioned only for clarity.

Exceptions

Cannot satisfy an item? File an exception — exceptions are normal, silent skips are not. The process (who approves, time-boxing rules, the form) is documented in Appendix C of this guide.

Section 1 — Universal Deployment Cycle

Applies to every project, every language, every stack.

1.1 Repository & Branching

Item	Description
Code repository	Lives in the company GitHub org; naming follows main policy Appendix E.
Three long-lived branches	develop (dev env), uat (staging/UAT), main (production). No other long-lived branches.
Feature branches	feature/<ticket>-<short-desc>, fix/<ticket>-<short-desc>, hotfix/<ticket>-<short-desc>; short-lived; PR into develop.
Promotion flow	feature → develop → uat → main, via PRs. No direct commits to develop, uat, or main.
Branch protection	develop, uat, main are all protected: required PR reviews, required passing CI, no direct pushes, no force-pushes.
PR hygiene	Small, focused PRs with ticket reference and at least one approval before merge.
Hotfix path	hotfix/* branches from main, merges to main with expedited review, then back-merged to uat and develop. [Prod only]
Conventional commits	Commits follow a standard format (feat:, fix:, chore:) for clean release notes.
No large binaries	No archives, build outputs, or media in the repo; use object storage or Git LFS if needed.

1.2 Documentation

Item	Description
README.md	Project description, setup, build/run/test, env overview, architecture link, deploy notes, contacts.
.env.example	Placeholder values for every required variable; updated whenever a new variable is added.
Architecture diagram	Diagram-as-code source plus rendered image in /docs, depicting the production topology. Dev and UAT are scaled-down references against this diagram. Updated whenever production topology changes.
Runbook	Deploy, rollback, common failures, where to find logs/metrics/alerts, escalation contacts. [Prod only]
API documentation	OpenAPI / Swagger spec committed and served at a documented endpoint in every environment. If APIs vary between dev / UAT / prod (e.g., test endpoints exposed in non-prod), each environment's docs reflect what is actually deployed there.

1.3 Secrets & Configuration

Item	Description
No secrets in code	Zero hardcoded credentials, keys, or tokens anywhere — including tests and comments.
Config from environment	All config read from env vars or a secret manager at runtime; nothing baked into images.
Secret manager	Secrets stored in AWS Secrets Manager / GCP Secret Manager / Vault for every environment (dev, UAT, prod) — not just production. Local-only .env files are for local developer machines and never reach a deployed environment.
Multi-environment config	The same code image runs in dev / UAT / prod by reading environment-specific values (DB host, API keys, feature toggles) at startup from env vars or a secret manager. No environment-specific values baked into the image.
Env validation at startup	A small script (e.g., validate_env.py — see Appendix A.1) runs when the container starts and fails fast with a clear error naming any missing or malformed required env vars. Prevents the container from starting in a half-broken state.
Secret rotation	Rotation per main policy Appendix G; app reloads without code changes. [Prod only]

1.4 Containerization (Mandatory)

Mandatory — every application is dockerized.

Every service the company builds is delivered as a container image. Bare-VM, host-installed, or interpreter-launched deployments are not permitted.

Item	Description
Multi-stage Dockerfile	Build and runtime stages separated; runtime contains only what's needed (Appendix A.2). Same Dockerfile used across dev, UAT, prod.
Pinned base image	Pinned to a minor version (never :latest) in every environment; production additionally pins by SHA256 digest.
Non-root user	Container runs as a non-root user created in the Dockerfile, in every environment.
.dockerignore	Excludes .git, build outputs, caches, .env, IDE files, test artifacts.
docker-compose.yml	Local-dev only; pinned versions for DB / cache / broker; volumes for persistence.
Image size	The final image is reasonable for the stack; bloat indicates leftover build artifacts. Checked at build time, applies to every environment.
Image scanning	Trivy image scan (trivy image) runs before every deploy — dev, UAT, prod. Critical findings block.

Item	Description
HEALTHCHECK declared	Dockerfile HEALTHCHECK exercises the application (not just process liveness). Same declaration applies in dev, UAT, prod.
Image registry	Published to the project container registry (ECR / Artifact Registry / ACR), tagged with commit SHA and environment.

1.5 Observability

Item	Description
Health endpoint	/health returns 200 when alive; /ready returns 200 only when dependencies are reachable.
Logs to STDOUT/STDERR	App writes to standard streams; no log files inside containers.
Structured logs	JSON with consistent fields: timestamp, level, service, request_id, message.
Log rotation (host-level)	Handled by container runtime / Kubernetes / journald — never by the application.
Correlation IDs	Trace ID propagated through downstream calls and present in every log line.
Metrics endpoint	Exposes request rate, error rate, latency percentiles, key business metrics.
Dashboards	Grafana / New Relic dashboards exist; linked from the runbook. [Prod only]
Alerts	Configured on errors, latency, health failure, dependency outage; each linked to a runbook entry. [Prod only]
Log / Metrics Retention	Log / Metrics Retention Retention is set per environment, not per project. Defaults: • Application logs: dev 7d, UAT 14d, prod 30d hot + 1y cold. • Metrics: dev/UAT 14d, prod 90d hot + 1y downsampled. • Audit logs: prod 1y minimum, tamper-evident store. • Access logs (LB / CDN): prod 90d. Enforced via observability platform lifecycle rules and S3 lifecycle policies — never ad-hoc deletion. PII / PCI / PHI projects may need longer; agreed at G2.

1.6 Security Baseline

Item	Description
SonarQube quality gate	Project registered; gate is non-bypassable; failing gate blocks merge.
Dependency scan	Trivy filesystem scan (trivy fs) runs in CI against lock files; Critical & High block merge unless risk-accepted.
Secret scan	GitHub secret scanning + pre-commit; detected secrets block merge and trigger rotation.
Image scan	Container image scanned in pipeline by trivy image before every deploy — dev, UAT, prod. Critical findings block.
HTTPS / TLS only	All traffic over TLS in every environment (dev, UAT, prod) — HTTP redirects to HTTPS; internal service-to-service calls use TLS where supported. Self-signed certs acceptable in dev, real certs in UAT and prod.
Security headers	HSTS, CSP, X-Content-Type-Options, X-Frame-Options, Referrer-Policy verified by scanner.
Input validation	Inputs validated at the boundary; parameterized queries; output encoded.
AuthN / AuthZ	Authentication (proving who you are: login, tokens, certs) AND authorization (proving you are allowed to do this specific thing: role / permission checks) enforced at every protected endpoint — not just at login. Roles and permissions documented.
Rate limiting	Public endpoints rate-limited (especially auth and writes); limits documented. [Prod only]
Request size limits	Maximum body size enforced at proxy and app layers to prevent memory exhaustion.
OSS License Compliance	All third-party dependencies must use permitted licenses. Permitted by default: MIT, Apache 2.0, BSD (2-clause, 3-clause), ISC. Requires DevOps + Engineering review: LGPL, MPL. Not permitted without explicit approval: GPL, AGPL, SSPL, Commons Clause, or any "source-available but not OSS" license. License scan runs in CI (trivy fs covers this); a new dependency with a non-permitted license blocks merge.

1.7 Testing & Coverage

Item	Description
Unit tests	Cover new and changed code; run in CI; must pass before merge.
Integration tests	Cover critical flows (auth, payment, ingestion, integrations); run in CI.
Code coverage	Reported in CI; minimum 70% on changed code, 60% overall (adjustable per project).
Smoke tests	Cadence, depth, and target environments defined in the project's test plan. At minimum: a happy-path smoke test runs against UAT after each deployment. [Per project plan]
Load test (G5)	UAT load test results documented; feed the refined cost estimate. [Prod only]

Item	Description
No skipped tests on develop/uat/main	Tests not skipped or commented out; fix or remove, do not silently disable.

1.8 CI/CD Pipelines — Triggers & Stages

Application repo and Terraform repo run separate pipelines.

The application repo (this section) builds, tests, scans, and deploys application images. The Terraform repo (Section 1.11) plans and applies infrastructure changes. Both repos are protected the same way — PR review, branch protection, mandatory CI.

Application repository pipeline

Trigger	Stages (in order)
PR opened against develop, uat, or main	1. Lint (language-specific) 2. Unit + integration tests 3. Code coverage report (threshold enforced) 4. SonarQube SAST scan (non-bypassable quality gate) 5. Dependency vulnerability scan (trivy fs) 6. Secret scan (gitleaks / GitHub native) 7. Build container image 8. Image vulnerability scan (trivy image) 9. Post a summary comment to the PR No deploy on PR — deploy only happens on merge.
Merge to develop	1. Re-run full PR pipeline against merge commit 2. Tag image with commit SHA + 'develop' tag 3. Push to container registry 4. Auto-deploy to dev environment (Terraform-provisioned) 5. Run smoke tests against dev 6. Notify team channel of deploy result
Merge to uat	1. Re-run full pipeline 2. Tag image with commit SHA + 'uat' tag 3. Push to container registry 4. Auto-deploy to UAT (Terraform-provisioned) 5. Run smoke tests against UAT 6. Notify team channel of deploy result
Merge to main	1. Re-run full pipeline 2. Tag image with commit SHA + version tag + 'prod' tag 3. Push to container registry 4. Pipeline pauses at a manual approval gate 5. On approval: deploy to production 6. Run smoke tests against production 7. Notify team channel of deploy result

Application repo CI requirements

Item	Description
Pipeline as code	Jenkinsfile or GitHub Actions workflow committed; no UI-only configuration.
All scan stages mandatory	Lint, SonarQube, dependency scan, secret scan, image scan are non-bypassable. Failing any blocks merge.
Reproducible builds	Same source produces the same artifact; dependencies pinned; image tagged with commit SHA.
Manual approval to prod	Merge to main does not auto-deploy. Pipeline pauses for explicit human approval before production deploy. [Prod only]
Rollback path	Pipeline supports redeploying a previous image tag in one action; runbook covers procedure. [Prod only]
PR status reporting	CI posts a summary to the PR with test results, coverage, scan findings, image size.

1.9 Pre-commit Hooks

Mandatory locally AND mirrored in CI.

Every developer installs pre-commit hooks locally on first clone (CONTRIBUTING.md documents the steps). The same checks also run as a CI stage on every PR as a safety net — so a missed local install does not bypass the standard.

Item	Description
.pre-commit-config.yaml	Committed to the repo; CONTRIBUTING.md tells new devs how to install (Appendix A.6).
Linters / formatters	Language-specific: ruff/black (Python), prettier+eslint (Node), spotless (Java).
Secret detection	detect-secrets / gitleaks runs on every commit.
Large file guard	Hook rejects files over a threshold (e.g., 5 MB).
Hygiene hooks	Trailing whitespace, EOF newline, mixed line endings, merge-conflict markers.

1.10 Deployment Readiness

Item	Description
Database migrations	Committed, idempotent, tested at each environment before promotion. Forward-and-back compatible across one release so a rollback does not require a data rollback.
Migration runs in pipeline	Migrations run as a deploy step in every environment (not manually); migration logs captured.
Backwards-compatible API changes	Removing or breaking an API field or DB column happens in two releases (deprecate, then remove) — applied at every promotion, not just at prod.
Feature flags for risky changes	Risky behavior ships behind a flag, default off in prod. [Prod only]
Documented rollback plan	Procedure written before deploy: how to revert code and (if needed) data. [Prod only]
Communication plan	Stakeholders notified; status page updated; comms drafted for known impacts. [Prod only]

1.11 Infrastructure — Terraform Repository

Mandatory — all infrastructure is provisioned via Terraform.

No production resource is created by hand. Console / CLI clicks are limited to break-glass exceptions only. Manually-created resources are tagged ManagedBy=manual, tracked under exception, and brought under Terraform on the next change.

Repository layout

Each application has its own Terraform repository: **<project>-infra**. Application code and infrastructure code live in separate repositories so they have independent review, independent change history, and independent blast radius.

```

<project>-infra/
├── environments/
│   ├── dev/
│   │   ├── main.tf           # calls modules with dev inputs
│   │   ├── terraform.tfvars  # dev-specific values (non-secret)
│   │   ├── backend.tf        # remote state config for dev
│   │   └── providers.tf
│   ├── uat/
│   │   ├── main.tf
│   │   ├── terraform.tfvars
│   │   ├── backend.tf
│   │   └── providers.tf
│   └── prod/
│       ├── main.tf
│       ├── terraform.tfvars
│       ├── backend.tf
│       └── providers.tf
├── modules/
└── network/                  # VPC, subnets, routes, gateways

```

```

├── compute/      # ECS / EKS / EC2 / Lambda
├── database/     # RDS, parameter groups, backups
├── storage/      # S3 buckets, lifecycle rules
├── iam/          # roles, policies
├── observability/ # log groups, alarms, dashboards
├── .github/workflows/ # or Jenkinsfile
├── .gitignore    # excludes .terraform/, *.tfstate*
├── .pre-commit-config.yaml # terraform_fmt, terraform_validate, tfsec
└── README.md

```

Terraform repo CI/CD

Trigger	Stages (in order)
PR opened against develop, uat, or main	1. terraform fmt -check 2. terraform validate (per environment) 3. tfllint 4. tfsec (IaC security scan) 5. Checkov (compliance scan) 6. terraform plan (against the target environment based on the PR target branch) 7. Post the plan output as a comment on the PR No apply on PR.
Merge to develop	1. Re-run fmt / validate / tfllint / tfsec / Checkov 2. terraform plan against dev 3. Pipeline pauses at manual approval gate 4. On approval: terraform apply against dev
Merge to uat	1. Same scan and plan stages 2. Pipeline pauses at manual approval gate 3. On approval: terraform apply against UAT
Merge to main	1. Same scan and plan stages 2. Pipeline pauses at manual approval gate 3. On approval: terraform apply against production

Terraform repo requirements

Item	Description
One Terraform repo per application	Named <project>-infra in the company GitHub org. Same branch model and protection as the application repo.
Environments / modules layout	environments/{dev,uat,prod}/ each call modules from modules/. No environment writes resources directly; everything goes through a module.
Branch protection (Terraform)	develop, uat, main protected; required PR review, required passing CI, no direct pushes, no force-pushes.
PR review same as application repo	At least one approver; reviewer must read the plan output posted by CI before approving.

Item	Description
Remote state with locking	State in S3 + DynamoDB (or GCS / Azure Blob with locking). One state per environment. Never committed to git.
Terraform version pinned	Pinned in required_version (e.g., '~> 1.8.0'). Same version in dev, CI, and developer machines (tfenv recommended).
Provider versions pinned	Pinned in required_providers with version constraints. Lockfile (.terraform.lock.hcl) committed.
Plan posted to PR	terraform plan output posted as a PR comment so reviewers can see exactly what will change.
Manual approval to apply	Apply is always gated by manual approval — even in dev. This is the deliberate, documented choice.
tfsec / Checkov gates	High-severity IaC findings block merge unless risk-accepted by DevOps in writing.
No secrets in tfvars	terraform.tfvars contains non-secret values only. Secrets come from environment variables (TF_VAR_*) or a secret manager.
Tagging applied via modules	Every resource carries the required tags (main policy Appendix D). Enforced at the module level so it cannot be omitted.
Module versioning	If modules are shared between repos, they are versioned (semver) and each environment pins to a module version.

1.12 Access Management

No direct access to server resources for developers. All servers, databases, and cloud resources are managed by the DevOps team. For debugging, logs and dashboards are available via the observability tools (read-only). If deeper access is needed (live DB query, shell on a host, config inspection), connect with the DevOps team — they assist or perform the action on your behalf.

Section 2 — Python / Django Addendum

In addition to Section 1.

Item	Description
requirements.txt / pyproject.toml	Pinned versions; lock file (requirements.lock, poetry.lock, uv.lock) committed.
Reproducible install	CI and Docker install from the lock file (--frozen / --frozen-lockfile).
Virtual environment	Local dev uses venv / poetry / uv; never installed into system Python.
Entry point	Clear entry point (manage.py, app.py, main.py); documented in README.
Production WSGI / ASGI server	Gunicorn or Uvicorn / Hypercorn — never runserver / flask run (Appendix A.3).
Settings per environment	Either split files (settings/base.py + dev.py + uat.py + prod.py) or a single settings.py driven by env vars. The same code runs in every environment; only inputs change. DJANGO_SETTINGS_MODULE (or equivalent) set per environment.
DEBUG=False outside dev	DEBUG may be True in local dev only. DEBUG is False in deployed environments (dev, UAT, prod) with ALLOWED_HOSTS set to the actual hostnames, never '*'.
Secure cookies and CSRF	SESSION_COOKIE_SECURE, CSRF_COOKIE_SECURE, SECURE_SSL_REDIRECT, HSTS, CSRF_TRUSTED_ORIGINS.
CORS configured	django-cors-headers with explicit allowed origins in every deployed environment. No wildcard '*' anywhere. Dev / UAT origins listed alongside production.
Static files (collectstatic)	Runs at deploy; served via Nginx / CDN / S3 — not by Django.
Media files	User uploads in object storage (S3 / GCS); direct-to-storage uploads where possible.
Database connection pooling	CONN_MAX_AGE / pgbouncer / framework pool tuned to backend capacity.
Celery / async workers	Workers in their own containers with their own scaling and observability.
validate_env script	Runs at container start and fails fast on missing env vars (Appendix A.1).
Timezone = UTC	TIME_ZONE = 'UTC', USE_TZ = True; convert at the presentation layer.
Logging configured	JSON logs to stdout in every deployed environment (Appendix A.10). django.db.backends and urllib3 tuned to WARNING to avoid noise. The same logging configuration runs in dev / UAT / prod.

Section 3 — Node.js / React Addendum

In addition to Section 1.

Item	Description
package.json	Scripts for start, dev, build, test, lint; engines field pins Node major version.
Lock file committed	package-lock.json / yarn.lock / pnpm-lock.yaml; CI uses npm ci / --frozen-lockfile.
Node version pinned	.nvmrc or volta config; same Node version in dev, CI, and Dockerfile.
Dockerfile uses npm ci	Copy package.json + lock file first, then npm ci, then source — for layer caching.
Multi-stage build	Build stage installs all deps and builds; runtime installs prod deps only (Appendix A.4).
Production deps only at runtime	Runtime image excludes devDependencies (npm ci --omit=dev).
Optimized build (frontend)	Production build minifies, tree-shakes, code-splits, content-hashes filenames.
Static asset hosting	Static assets served via CDN or object storage — not from the Node process.
Process manager	Production runs Node directly or via PM2 / orchestrator — never nodemon.
Graceful shutdown	SIGTERM stops new connections, drains in-flight, then exits (Appendix A.9).
Error handling	Error middleware in place; unhandled rejections logged and trigger graceful shutdown.
Helmet / security headers	Helmet (Express) or framework equivalent for security headers.
TypeScript strict mode	If TypeScript: strict: true; suppressed errors require comment justification.
Bundle size budget	Frontend bundle size monitored per route / chunk; regressions flagged in PR.
.dockerignore excludes node_modules	node_modules always installed inside the build, never copied from host.

Section 4 — Java / Spring Addendum

In addition to Section 1.

Item	Description
Build tool & version	Maven / Gradle wrapper committed (mvnw / gradlew); identical build in dev, CI, Docker.
Dependency versions pinned	Managed via BOM or explicit versions; no ranges; no SNAPSHOT in releases.
Java version pinned	Declared in build file; matches Dockerfile base image; LTS (17 / 21) preferred.
Reproducible build	Deterministic output; CI build runs offline-capable.
Multi-stage Dockerfile	JDK build stage; JRE / jlink runtime stage; layered Spring Boot JARs for caching.
JVM tuning	Container-aware flags; -XX:MaxRAMPercentage matches container memory; deliberate GC choice.
Profile per environment	Spring profiles dev / uat / prod via SPRING_PROFILES_ACTIVE.
Externalized config	Secrets and env-specific config from env vars / Vault — never in application.yml.
Actuator endpoints	/actuator/health and /actuator/info exposed in every environment. Sensitive endpoints (/env, /heapdump, /threaddump, /loggers) are disabled or restricted to internal IPs in every deployed environment, including dev. Liveness and readiness exposed as separate health groups.
Code coverage (JaCoCo)	JaCoCo plugin configured; minimum threshold enforced as build failure.
Static analysis	SpotBugs / PMD / Checkstyle alongside SonarQube; format enforced via Spotless on pre-commit.
Logging (SLF4J + Logback)	SLF4J as the logging API. Logback configured for JSON output to stdout in every deployed environment. No System.out.println.
Graceful shutdown	server.shutdown=graceful; spring.lifecycle.timeout-per-shutdown-phase set.
Connection pool tuned	HikariCP is sized to backend capacity in every environment. Leak-detection threshold enabled (it surfaces leaks earlier in dev/UAT, but is left on in prod as well).
Metrics (Micrometer)	Wired to Prometheus / New Relic; custom business metrics where useful.

Section 5 — Other Stacks (Generic Addendum)

For Go, Ruby, .NET, PHP, Rust, etc. Tools differ; requirements do not.

Item	Description
Pinned dependencies + lock file	go.sum / Gemfile.lock / packages.lock.json / composer.lock / Cargo.lock committed; CI installs from lock.
Reproducible build	Toolchain version pinned (go.mod toolchain, .ruby-version, global.json, rust-toolchain.toml).
Multi-stage Dockerfile	Build / runtime split; minimal runtime image (distroless / alpine).
Production server / runtime	Go: direct binary; Ruby: Puma; .NET: Kestrel + reverse proxy; PHP: PHP-FPM + Nginx.
Config from environment	The same binary or image runs in every env; only inputs change.
Startup validation	Validate required config at startup; fail fast with a clear error.
Graceful shutdown	SIGTERM stops new work, finishes in-flight, then exits.
Health and readiness endpoints	Documented port and path; used by load balancer and orchestrator.
Structured logs to stdout	JSON or key-value to stdout/stderr; no log files written by the app.
Code coverage tooling	go test -cover / simplecov / coverlet / phpunit-coverage / cargo-tarpaulin reported in CI.

Section 6 — Production Hardening

Required for medium / large projects and any project handling sensitive data (PII / PCI / PHI). Small internal projects may defer with a documented exception.

[Prod only] — this entire section applies to the production environment.

Items in this section are not required at dev or UAT (though some — like resource limits and network policies — are often easier to set up correctly from the start in UAT as a rehearsal). To keep rows uncluttered, individual items are not separately tagged.

Item	Description
Resource limits	CPU and memory limits set in the orchestrator (K8s requests/limits, ECS task definition).
Read-only root filesystem	Container root is read-only; writable mounts only where needed (e.g., /tmp).
Image digest pinning	Production references images by SHA256 digest, not tag.
Network policies	Egress restricted; ingress through load balancer / service mesh only.
Backup verification	Backups restored to a test env at least quarterly — untested backup ≠ backup.
DR posture documented	RPO / RTO defined per env; DR procedure rehearsed at least annually.
Feature flags	Flag system in place; flags default off in prod; each has owner and removal date.
API versioning	External APIs versioned (/v1/, /v2/) with documented deprecation windows.
Idempotent endpoints	Retryable writes accept an idempotency key to prevent double-processing.
Circuit breakers / timeouts	Every outbound call has a timeout; critical paths have circuit breakers.
Chaos / failure testing	Failure modes exercised in UAT (dep down, slow dep, DB failover, pod kill); required for large.
Audit logging	Security-relevant events (login, perm change, data access, admin) to tamper-evident store.
Data retention policy	Retention period documented per data class; automated purge in place.
PII handling	PII inventoried, encrypted at rest and in transit, redacted in logs, access audited.

Appendix A — Code & Config Reference

Starting points, not finished code. Adapt to your project. Each is referenced from a checklist item above.

A.1 Environment Variable Validation (Python)

Run at container start. Fails fast on missing variables — better than failing later inside a request handler.

```
# validate_env.py
import os, sys

REQUIRED = [
    "DB_HOST", "DB_USER", "DB_PASSWORD", "DB_NAME",
    "SECRET_KEY", "REDIS_URL", "ALLOWED_HOSTS",
]
OPTIONAL_WITH_DEFAULT = {"LOG_LEVEL": "INFO", "PORT": "8000"}

def main():
    missing = [v for v in REQUIRED if not os.getenv(v)]
    if missing:
        print(f"FATAL: missing env vars: {' '.join(missing)}", file=sys.stderr)
        sys.exit(1)
    for k, default in OPTIONAL_WITH_DEFAULT.items():
        os.environ.setdefault(k, default)
    print("env validation: OK")

if __name__ == "__main__":
    main()
```

A.2 Multi-stage Dockerfile (Python)

```
# syntax=docker/dockerfile:1.7
FROM python:3.11.9-slim AS builder
WORKDIR /build
RUN pip install --no-cache-dir uv
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev

FROM python:3.11.9-slim AS runtime
RUN groupadd -r app && useradd -r -g app app
WORKDIR /app
COPY --from=builder /build/.venv /app/.venv
COPY --chown=app:app . /app
ENV PATH="/app/.venv/bin:$PATH" \
    PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1
USER app
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 \
    CMD python -c "import urllib.request,sys; \
        sys.exit(0 if urllib.request.urlopen('http://localhost:8000/health').status==200 else 1)"
ENTRYPOINT ["sh", "-c", "python validate_env.py && exec gunicorn -c gunicorn.conf.py \
    myproject.wsgi:application"]
```

A.3 Gunicorn Production Config

```
# gunicorn.conf.py
import multiprocessing, os

bind = f"0.0.0.0:{os.getenv('PORT', '8000')}"
workers = int(os.getenv("GUNICORN_WORKERS", multiprocessing.cpu_count() * 2 + 1))
worker_class = "gthread"
threads = int(os.getenv("GUNICORN_THREADS", "4"))
timeout = 30
graceful_timeout = 30
keepalive = 5
max_requests = 1000
max_requests_jitter = 100
accesslog = "-"
errorlog = "-"
loglevel = os.getenv("LOG_LEVEL", "info").lower()
```

A.4 Multi-stage Dockerfile (Node.js)

```
# syntax=docker/dockerfile:1.7
FROM node:20.13.1-alpine AS builder
WORKDIR /build
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build && npm prune --omit=dev

FROM node:20.13.1-alpine AS runtime
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY --from=builder --chown=app:app /build/node_modules ./node_modules
COPY --from=builder --chown=app:app /build/dist ./dist
COPY --from=builder --chown=app:app /build/package.json ./
USER app
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 \
  CMD wget -qO- http://localhost:3000/health || exit 1
CMD ["node", "dist/server.js"]
```

A.5 .dockerignore

```
.git
.gitignore
.github
.idea
.vscode
*.swp
.env
.env.*
!.env.example
```

```

dist
build
target
*.egg-info
__pycache__
*.pyc
.pytest_cache
.mypy_cache
.ruff_cache
.coverage
htmlcov
node_modules
.next
.nuxt
.gradle
logs
*.log
.DS_Store
Thumbs.db
tests
docs
README.md

```

A.6 .pre-commit-config.yaml

```

repos:
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.6.0
  hooks:
    - id: trailing-whitespace
    - id: end-of-file-fixer
    - id: check-merge-conflict
    - id: check-added-large-files
      args: [--maxkb=5000]
    - id: detect-private-key
    - id: check-yaml
    - id: check-json

- repo: https://github.com/gitleaks/gitleaks
  rev: v8.18.4
  hooks:
    - id: gitleaks

# Python
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.5.0
  hooks:
    - id: ruff
      args: [--fix]
    - id: ruff-format

# Terraform
- repo: https://github.com/antonbabenko/pre-commit-terraform
  rev: v1.92.0
  hooks:
    - id: terraform_fmt
    - id: terraform_validate
    - id: terraform_tfsec

```

A.7 Trivy — Dependency & Image Scanning

Trivy is the single tool used for both dependency scanning and container image scanning. The two CI stages call it in different modes:

```
# 1. Dependency scan — runs early in CI against the source tree.
# Scans lock files (requirements.lock, package-lock.json, pom.xml,
# go.sum, Gemfile.lock, etc.) for known vulnerabilities.
trivy fs \
  --severity HIGH,CRITICAL \
  --exit-code 1 \
  --ignore-unfixed \
  --scanners vuln,secret,misconfig \
  .

# 2. Image scan — runs after the image is built, before publishing/deploying.
# Scans the OS packages and application layers inside the image.
trivy image \
  --severity HIGH,CRITICAL \
  --exit-code 1 \
  --ignore-unfixed \
  myregistry.example.com/${PROJECT}:${COMMIT_SHA}

# 3. (Optional) Generate a SARIF report and upload to GitHub code scanning.
trivy image \
  --format sarif \
  --output trivy-results.sarif \
  myregistry.example.com/${PROJECT}:${COMMIT_SHA}
```

Notes: the `--exit-code 1` flag is what makes a finding fail the pipeline. `--ignore-unfixed` skips findings that have no upstream fix available; remove it if your policy requires reviewing those too. Risk-accepted CVEs go in a `.trivyignore` file at repo root, one CVE per line, each with a comment explaining why it was accepted and an expiry date for re-review.

A.8 Health vs Readiness Endpoints

Liveness answers: is the process alive? Cheap, no external calls. Failure → orchestrator restarts the container. **Readiness** answers: am I ready to serve traffic? Checks dependencies. Failure → orchestrator stops sending traffic but does not restart.

```
def health():          # liveness
  return {"status": "ok"}, 200

def ready():           # readiness
  try:
    db.execute("SELECT 1")
    cache.ping()
  except Exception as e:
    return {"status": "degraded", "error": str(e)}, 503
  return {"status": "ready"}, 200
```

A.9 Graceful Shutdown (Node.js)

```
const server = app.listen(port);

function shutdown(signal) {
  console.log(`Received ${signal}, draining connections...`);
  server.close((err) => {
    if (err) { console.error(err); process.exit(1); }
    process.exit(0);
  });
  setTimeout(() => process.exit(1), 30_000).unref();
}

process.on("SIGTERM", () => shutdown("SIGTERM"));
process.on("SIGINT", () => shutdown("SIGINT"));
```

A.10 Structured Logging to STDOUT (Django)

```
# settings/prod.py
LOGGING = {
  "version": 1,
  "disable_existing_loggers": False,
  "formatters": {
    "json": {
      "()": "pythonjsonlogger.jsonlogger.JsonFormatter",
      "format": "%(asctime)s %(levelname)s %(name)s %(message)s",
    },
  },
  "handlers": {
    "stdout": {
      "class": "logging.StreamHandler",
      "stream": "ext://sys.stdout",
      "formatter": "json",
    },
  },
  "root": {"handlers": ["stdout"], "level": "INFO"},
  "loggers": {
    "django.db.backends": {"level": "WARNING"},
    "urllib3": {"level": "WARNING"},
  },
}
```

Note on log rotation: do not rotate inside the application. Write to stdout and let the container runtime rotate. Application-side rotation conflicts with container log collectors and silently loses logs.

A.11 Terraform environment/main.tf (example)

```
# environments/dev/main.tf
terraform {
  required_version = "~> 1.8.0"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}
```

```

module "network" {
  source = "../../modules/network"
  project = var.project
  environment = "dev"
  cidr_block = var.vpc_cidr
}

module "compute" {
  source = "../../modules/compute"
  project = var.project
  environment = "dev"
  vpc_id = module.network.vpc_id
  subnet_ids = module.network.private_subnet_ids
  image_uri = var.app_image_uri
  cpu = 256
  memory = 512
}

module "database" {
  source = "../../modules/database"
  project = var.project
  environment = "dev"
  vpc_id = module.network.vpc_id
  subnet_ids = module.network.db_subnet_ids
  instance_class = "db.t4g.small"
}

```

A.12 Terraform backend.tf (example)

```

# environments/dev/backend.tf
terraform {
  backend "s3" {
    bucket = "company-tfstate-dev"
    key = "etailplus/dev/terraform.tfstate"
    region = "ap-south-1"
    dynamodb_table = "company-tfstate-locks"
    encrypt = true
  }
}

```

Appendix B — Per-Gate Quick Map

Items shown here must be verified at the named gate; items may be verified earlier.

Gate 4 (Build & Provision) — verify on first pipeline run

- Application repo created in company GitHub org with proper naming.
- Terraform repo (<project>-infra) created with environments/ + modules/ structure.
- Branch protection enforced on develop, uat, main in both repos.
- Pre-commit hooks installed and config committed in both repos.
- Application pipeline runs all mandatory stages: lint → test → SonarQube → trivy fs → secret scan → build → trivy image.
- Terraform pipeline runs all mandatory stages: fmt → validate → tflint → tfsec → Checkov → plan.
- SonarQube quality gate failing blocks merge.
- .env.example present and accurate; .dockerignore present.
- Multi-stage Dockerfile with non-root user and pinned base image.
- Image published to registry, tagged with commit SHA and environment.
- Dev and UAT environments provisioned via Terraform; all tags applied.
- Remote state with locking configured for Terraform.

Gate 5 (Pre-Production Readiness) — verify before prod cutover

- /health and /ready endpoints implemented and used by orchestrator.
- Structured logs to stdout; correlation IDs propagated.
- Metrics exposed; dashboards exist; alerts configured with runbook entries.
- DEBUG=False, ALLOWED_HOSTS set, secure cookies, HSTS, CORS allowlist (web apps).
- TLS enforced; security headers present.
- Rate limits and request size limits in place.
- Migrations idempotent and rerunnable; tested in UAT.
- UAT load test report available; refined cost estimate filed.
- Backup configured AND a restore tested in the last 30 days.
- Runbook published with rollback procedure.

Gate 6 (Go-Live) — verify on go-live call

- Production infrastructure provisioned via the same Terraform modules as UAT.
- All required tags applied in production via module-level enforcement.
- Production deploy gated by manual approval in pipeline (application).
- Production terraform apply gated by manual approval (infrastructure).
- Cost dashboard live; budget alerts at 50/80/100%.

- Cutover plan with explicit rollback criteria signed off.
- Communication plan executed (stakeholders, status page).
- Go/no-go decision recorded with named decision-makers.

Appendix C — Exception Process

If a checklist item cannot be satisfied — for legitimate reasons — file an exception rather than silently skipping it. The process is intentionally lightweight, but it is the only sanctioned way to deviate.

When to file

You hit a hard constraint that prevents satisfying an item (client mandate, third-party limitation, time-bound business deadline).

You believe an item is genuinely not applicable to your project (small internal tool, throwaway POC, prototype with no real users).

You need a temporary deferral while a permanent fix is in flight (an open ticket to make the dependency compliant, a vendor patch landing next month).

What is not a valid reason: "I forgot," "it was easier not to," "we ran out of time" without a tracked follow-up.

Approval:

Email approval from the Head of Engineering and the DevOps team. That's the whole process.

Document Control

Version History

Version	Date	Author	Change Summary
1.0	May 25, 2026	DevOps Team	Initial release alongside DevOps Engagement & Governance Policy v1.0.
1.1	May 25, 2026	DevOps Team	Condensed: descriptions reduced to one line each.
1.2	May 25, 2026	DevOps Team	Reframed as end-to-end deployment cycle guide. Branching model updated to develop/uat/main. CI pipelines specified per trigger. Mandatory Docker callout added. New Section 1.11 covering the Terraform repository (one per app, environments/+modules/ layout, plan-on-PR / manual-apply).
1.3	May 25, 2026	DevOps Team	Dependency vulnerability scan tool standardized on Trivy (trivy fs); image scan standardized on Trivy (trivy image). New Appendix A.7 with Trivy command reference and .trivyignore guidance. Checklist column renamed from 'Verify by' to 'Required for'.
1.4	May 25, 2026	DevOps Team	Dropped the per-item stage column; replaced with inline colored tags [UAT onwards] and [Prod only] used only where genuinely stage-specific (most items now apply universally). Reclassified items per stakeholder review: Secret manager, multi-env config, env validation, image scanning, HEALTHCHECK, HTTPS/TLS, migrations, BC API changes are all-envs. Architecture diagram clarified as production-topology reference. Smoke tests tagged [Per project plan] with UAT minimum floor. Section 6 declared [Prod only] at section level. Pre-commit clarified as mandatory locally AND mirrored in CI. Plain-language terminology callout added. On-call assignment removed.
1.5	May 25, 2026	DevOps Team	Removed the [UAT onwards] tag from every row across Section 1 (Containerization, Observability, Security baseline, Testing, CI/CD, Deployment readiness, Terraform) and all stack sections (Python/Django, Node.js/React, Java/Spring, Other). Tag is no longer used in the document — almost every item applies from dev day one. Rewrote six

Version	Date	Author	Change Summary
			descriptions (DEBUG, CORS, Settings per env, Logging, HikariCP, Actuator) to encode environment-conditional behavior that the tag had previously carried implicitly. Only [Prod only] and [Per project plan] tags remain.

Review Cadence

- Reviewed every 6 months in lockstep with the main policy.
- Updates may be proposed any time by any developer via PR to the document repository.
- Material changes require approval from the DevOps lead.